CYBERAWARE

CYBERSECURITY AWARENESS PLATFORM

MERN Stack Project Report

Submitted in partial fulfilment of the academic project requirements

Student Name(s)	M.LAVANYA P.AKSHAYA P.SRINITHYA P.VANDHANA
Roll Number(s)	25MVCSMR0660 25MVCSMR0669 25MVCSMR0670 25MVCSMR0666
Department	CSE(AI&ML)
College	Malla Reddy Vishwavidhyapeeth
Academic Year	2026–2027

Table of Contents

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Scope of the Project

6. Target Users

7. Technologies Used

8. System Architecture

9. Main Modules and Features

10. Functional Requirements

11. Non-Functional Requirements

12. Database Design

13. REST API Design

14. Authentication and Security

15. User Workflow

16. Testing Plan

17. Expected Output and Benefits

18. Limitations and Future Enhancements

19. Conclusion

20. References

1. Abstract

Cybersecurity Awareness Platform is a web-based learning application designed to help users understand common online threats and develop safer digital habits. The platform provides short learning modules, multiple-choice quizzes, immediate feedback, explanations, and a record of quiz results. Administrators can manage learning modules and quiz questions, while learners can study topics and practise their knowledge.

The proposed full-stack implementation uses MongoDB to store users, learning modules, quiz questions, and quiz attempts. Express.js and Node.js provide REST API endpoints and server-side validation, while React.js presents a responsive user interface. Mongoose defines and validates database models, and JSON Web Tokens (JWT) are used for authenticated API access. Role-based middleware separates administrator functions from learner functions.

2. Introduction

People use digital services for education, communication, shopping, banking, and work. These activities can expose them to phishing messages, weak passwords, malicious links, unsafe downloads, and other security risks. Many incidents can be reduced when users recognize warning signs and follow basic security practices.

The Cybersecurity Awareness Platform offers a simple place to learn, practise, and review cybersecurity concepts. Lessons explain practical safety habits in accessible language. Quizzes help users check their understanding, and saved results encourage continuous learning. The project also demonstrates how a modern JavaScript technology stack can be used to build a database-driven web application.

3. Problem Statement

Users may lack an accessible and structured way to learn cybersecurity basics and test whether they understand safe online behaviour. Static information alone may not provide practice or measurable feedback. The project addresses this problem by combining learning modules, interactive quizzes, explanations, score tracking, and content management in one web application.

4. Objectives

- Demonstrate full-stack development using the MERN stack: MongoDB, Express.js, React.js, and Node.js.
- Implement CRUD (Create, Read, Update, Delete) operations for learning modules and quiz questions.
- Build REST APIs for authentication, learning content, quiz questions, and results.
- Provide an accessible React interface for lessons, quizzes, and learner progress.
- Use authentication and role-based authorization to restrict administrative actions.
- Calculate quiz scores on the server and save attempts for later review.
- Promote awareness of phishing, password security, multi-factor authentication, safe browsing, and device security.

5. Scope of the Project

The platform is intended as an educational project prototype. Learners can browse modules, answer quiz questions, receive results, and view saved attempts after signing in. Administrators can add, edit, and delete learning content and quiz questions. The application can be expanded with additional topics, question banks, analytics, and user-management functions.

The current source package provides a MERN implementation and a seed script for initial sample content. The hosted demonstration created during development is a separate lightweight demo and should not be confused with the MongoDB-backed MERN implementation. The source package should be installed and tested in a local or hosted environment before formal submission.

6. Target Users

- Learners and students who want to improve their cybersecurity knowledge.
- Teachers or trainers who want to share introductory cybersecurity lessons.
- Administrators who maintain learning modules and quiz questions.

6. Technologies Used

Frontend — React.js: builds the interactive user interface, lesson cards, quiz flow, forms, and results display.

Backend — Node.js: runs the JavaScript server environment.

API framework — Express.js: defines REST endpoints and middleware for request handling, authentication, and authorization.

Database — MongoDB: stores user accounts, learning modules, questions, and quiz attempts.

ODM — Mongoose: defines schemas, data types, validation rules, and database operations.

Authentication — bcryptjs and JSON Web Tokens (JWT): password hashing and token-based API authentication.

Development tools — npm and Vite: package management and local React development server.

8. System Architecture

The application follows a client–server architecture. A learner interacts with the React frontend. The frontend sends HTTP requests to Express REST APIs. The backend validates the request, checks authentication and role permissions where required, and uses Mongoose to read or write data in MongoDB. The API returns JSON responses, which React uses to update the page.

Data flow: User → React interface → Express/Node.js API → Mongoose models → MongoDB → JSON response → React interface.

Administrator-only endpoints require a valid bearer token and the administrator role. Quiz answers are evaluated by the backend against stored correct answers so that the client does not determine the authoritative score.

9. Main Modules and Features

1. Learning Modules: Displays lesson title, category, difficulty level, estimated duration, summary, and lesson content. The administrator can create, update, and delete modules.

2. Quiz Management: Stores a topic, question, four answer options, the correct option, and an explanation. The public question-list endpoint omits the correct-answer field to avoid exposing answers before submission. Administrators can add, update, and delete questions.

3. Quiz Attempt and Scoring: Learners select answers and submit them to the server. The backend checks the answers, calculates the score and percentage, and saves the attempt.

4. Results and Progress: Signed-in learners can view their recent saved attempts. Administrators can access a broader list of attempts for monitoring.

5. Authentication and Roles: The login endpoint verifies a password hash and returns a signed JWT. Middleware protects authenticated routes and restricts content-management operations to administrators.

6. Seed Data: A seed script creates an initial administrator account, a sample password-security lesson, and sample questions on phishing, passwords, multi-factor authentication, safe browsing, and device security.

10. Functional Requirements

- The system shall display available learning modules and their details.
- The system shall allow authorized administrators to create, update, view, and delete learning modules.
- The system shall display quiz questions and answer options without exposing correct answers in the public question list.
- The system shall allow authorized administrators to create, update, view, and delete quiz questions.
- The system shall authenticate users using email and password.
- The system shall restrict administrative API operations to users with the administrator role.
- The system shall evaluate submitted quiz answers on the server and calculate a score and percentage.
- The system shall save quiz attempts for authenticated learners and allow them to view their own results.
- The system shall display useful validation and error messages when requests fail.

11. Non-Functional Requirements

Usability: Navigation, lesson content, and quiz options should be understandable to beginners.

Security: Passwords must be hashed; protected routes must verify JWTs; administrator permissions must be checked on the server; inputs must be validated.

Maintainability: Frontend, routes, middleware, and database models are separated into individual files.

Reliability: API errors should return appropriate status codes and clear messages.

Responsiveness: The interface uses responsive CSS layouts for desktop and smaller screens.

Performance: Database queries should return only the information needed, and result history is limited to a manageable number of records.

12. Database Design

User collection: name, email, passwordHash, role, and timestamps. Email is unique. Password hashes are excluded from ordinary query results.

Module collection: title, category, level, durationMinutes, summary, content, and timestamps.

Question collection: optional module reference, topic, question, four options, correctAnswer index, explanation, and timestamps.

Attempt collection: user reference, score, total, percentage, submitted answer records, correctness flags, and timestamps.

Relationships: one user can have many quiz attempts. A question may reference a learning module. An attempt contains answer records referencing the questions answered.

13. REST API Design

GET /api/health — checks that the API is running.

POST /api/auth/login — authenticates a user and returns a JWT and basic user details.

GET /api/modules — retrieves published learning modules. GET /api/modules/:id — retrieves one module.

POST /api/modules — creates a module (administrator only). PUT /api/modules/:id — updates a module (administrator only). DELETE /api/modules/:id — deletes a module (administrator only).

GET /api/questions — retrieves questions without correct-answer values. An optional topic query can filter the list.

POST /api/questions, PUT /api/questions/:id, DELETE /api/questions/:id — administrator-only question management.

POST /api/attempts — saves and scores a quiz attempt for the signed-in user.

GET /api/attempts/mine — retrieves the signed-in learner's own attempts. GET /api/attempts — administrator-only attempt listing.

14. Authentication and Security

Passwords are hashed using bcryptjs rather than stored as plain text. On login, the server compares the supplied password with the stored hash and issues a JWT when credentials are valid. The frontend sends the token in the Authorization: Bearer header for protected requests.

Role checks are performed in backend middleware; hiding an administrative section in the interface is not treated as a security boundary. The public question-list endpoint excludes the correct answer. The quiz submission endpoint recalculates correctness on the server using the database question records.

Security limitations: the sample administrator credentials are for local development only and must be changed before use. For production deployment, use HTTPS, strong environment secrets, rate limiting, secure token storage strategy, account registration and recovery controls, logging, and appropriate database access restrictions.

15. User Workflow

Learner workflow: Open the platform → browse a lesson → sign in → start a quiz → choose an answer for each question → submit answers → view the score → open My Results to review saved attempts.

Administrator workflow: Sign in with an administrator account → open Manage → add or maintain learning modules and questions → review existing content and quiz attempts.

Setup workflow: Configure MongoDB and environment variables → install backend dependencies → run the seed script → start the Express server → install frontend dependencies → start Vite → open the local development URL.

16. Testing Plan

The following checks should be performed after installing and running the project. These are recommended test cases, not a claim that all tests have already passed.

1. API health: GET /api/health returns a successful status.
2. Login success: valid demo administrator credentials return a token and role.
3. Login failure: incorrect credentials return an unauthorized response.
4. Public module access: module list and detail endpoints return stored data.
5. Authorization: unauthenticated or learner requests to administrator-only endpoints are rejected.
6. Module CRUD: administrator can create, update, retrieve, and delete a module.
7. Question privacy: public question results do not include the correctAnswer field.
8. Quiz scoring: correct and incorrect answers produce the expected score and percentage.
9. Result privacy: a learner can view their own attempts but cannot access another learner's history through the learner endpoint.
10. Validation: malformed IDs, missing required fields, invalid answer choices, and database errors return appropriate error responses.
11. Responsive UI: check navigation, lesson cards, forms, and quiz options on desktop and mobile widths.

17. Expected Output and Benefits

The expected outcome is a responsive web application that brings cybersecurity lessons and self-assessment into one place. Users can practise identifying common threats, learn why an answer is correct, and track their scores. Administrators can maintain the content without editing database records manually. The project demonstrates full-stack JavaScript development, database modelling, REST API design, CRUD operations, and role-based access control.

18. Limitations and Future Enhancements

Current limitations include a starter set of lessons and questions, a simple login workflow, and a development-oriented setup. The source code is a project starter and needs installation, integration, and runtime testing in the target environment. It does not currently provide a full registration flow, password reset, email verification, detailed analytics dashboard, or a formal learning-management workflow.

Future enhancements could include user registration and profile management, password reset, multi-language lessons, timed quizzes, topic-wise performance charts, badges and certificates, question randomization rules, accessibility improvements, administrator audit logs, additional security headers, rate limiting, automated tests, and deployment to a production hosting environment.

19. Conclusion

The Cybersecurity Awareness Platform proposes a practical way to teach cybersecurity fundamentals through structured lessons and interactive quizzes. Its MERN architecture separates the React frontend, Express/Node.js backend, and MongoDB data layer. CRUD APIs support content management, while authentication, role checks, and server-side scoring demonstrate important full-stack concepts. With complete installation and testing, the project can serve as a useful academic prototype and a foundation for a more comprehensive cybersecurity education service.

20. References

1. React documentation — https://react.dev/
2. Node.js documentation — https://nodejs.org/docs/
3. Express.js documentation — https://expressjs.com/
4. MongoDB documentation — https://www.mongodb.com/docs/
5. Mongoose documentation — https://mongoosejs.com/docs/
6. JSON Web Token introduction — https://jwt.io/introduction
7. OWASP Top 10 — https://owasp.org/www-project-top-ten/